SQL patterns you can remember

14 patterns, 4 families. Learn the question clue before the syntax.

THE 4-MOVE INTERVIEW ROUTINE

1. **SPOT THE CLUE**
   Ranking, counting, matching, or following a sequence?
2. **NAME THE PATTERN**
   Say it before coding: This is top N per group.
3. **WRITE THE SKELETON**
   Start with the key SQL tool; fill in tables later.
4. **CHECK THE TRAP**
   Ties, NULLs, row grain, gaps, or ordering.

FOUR MEMORY BUCKETS

WINDOW
Rank, compare, or accumulate WITHOUT collapsing rows.

JOIN / EXISTS
Connect records or test for a matching record.

GROUP / CASE
Collapse to one group or reshape categories into columns.

CTE / SEQUENCE
Build stages, streaks, cohorts, and hierarchies.

How to spot the right tool

Find the clue in the question. Then choose a likely SQL pattern.

1	**Top 3 or highest per EACH group?**	ROW_NUMBER / DENSE_RANK + PARTITION BY
2	**Total so far, previous, or next?**	SUM OVER / LAG / LEAD
3	**One row per group or category?**	GROUP BY / conditional SUM(CASE)
4	**No matching row, or any matching row?**	NOT EXISTS / EXISTS
5	**Same table playing two roles?**	SELF JOIN with two aliases
6	**Latest record per ID?**	ROW_NUMBER then filter rn = 1
7	**Consecutive dates or IDs?**	GAPS & ISLANDS
8	**First activity month and later behavior?**	DATE COHORTS
9	**Parent -> child -> grandchild?**	RECURSIVE CTE
10	**Hard query with multiple steps?**	CTE PIPELINE

Patterns 1 + 2

Window ranking / Top N by group - Read cue, recall SQL, test yourself.

01 Window ranking

WINDOWS

SPOT IT Rank people, products, or scores inside each group.

REMEMBER WINDOW ranks rows without collapsing them.

SQL STARTER

```
SELECT team, person, score,
       RANK() OVER (
         PARTITION BY team
         ORDER BY score DESC
       ) AS pos
FROM scores;
```

WATCH RANK skips numbers after ties; DENSE_RANK does not.

SELF TEST Which rank keeps equal scores tied?

02 Top N by group

WINDOWS

SPOT IT Find the best 3 or newest 5 within EACH group.

REMEMBER PARTITION resets the ranking for every group.

SQL STARTER

```
WITH ranked AS (
 SELECT team, person, score,
  ROW_NUMBER() OVER (
   PARTITION BY team ORDER BY score DESC
  ) AS rn FROM scores
) SELECT * FROM ranked WHERE rn <= 3;
```

WATCH Use DENSE_RANK to include all ties at the cutoff.

SELF TEST Why not use LIMIT 3 for top 3 per team?

Patterns 3 + 4

Running totals / Previous / next - Read cue, recall SQL, test yourself.

03 Running totals

WINDOWS

SPOT IT Show the total SO FAR for every date or transaction.

REMEMBER SUM OVER + ORDER BY = rolling accumulation.

SQL STARTER

```
SELECT user_id, sold_at, amount,
 SUM(amount) OVER (
  PARTITION BY user_id
  ORDER BY sold_at, sale_id
  ROWS BETWEEN UNBOUNDED PRECEDING
  AND CURRENT ROW) AS running_total
FROM sales;
```

WATCH Choose a stable order and explicit ROWS frame.

SELF TEST What changes with PARTITION BY user_id?

04 Previous / next

WINDOWS

SPOT IT Compare an event with the event immediately before or after.

REMEMBER LAG looks BACK; LEAD looks AHEAD.

SQL STARTER

```
SELECT user_id, event_date, value,
 LAG(value) OVER (
  PARTITION BY user_id
  ORDER BY event_date
 ) AS previous_value
FROM events;
```

WATCH Sort deterministically; first row has no prior row.

SELF TEST Which function compares today with yesterday?

Patterns 5 + 6

Self joins / Anti-joins - Read cue, recall SQL, test yourself.

05 Self joins

RELATIONSHIPS

SPOT IT One row refers to another row in the SAME table.

REMEMBER One table, two roles, two aliases.

SQL STARTER

```
SELECT e.name AS employee,
       m.name AS manager
FROM employees e
LEFT JOIN employees m
 ON e.manager_id = m.id;
```

WATCH LEFT JOIN retains workers without a manager.

SELF TEST Why alias employees twice?

06 Anti-joins

RELATIONSHIPS

SPOT IT Find customers with NO orders or items with NO matches.

REMEMBER NOT EXISTS means prove the match is absent.

SQL STARTER

```
SELECT c.customer_id
FROM customers c
WHERE NOT EXISTS (
 SELECT 1 FROM orders o
 WHERE o.customer_id = c.customer_id
);
```

WATCH NOT IN is unsafe if its subquery returns NULL.

SELF TEST Which clause finds customers who never ordered?

Patterns 7 + 8

Grouped aggregates / Conditional pivots - Read cue, recall SQL, test yourself.

07 Grouped aggregates

SUMMARIES

SPOT IT One answer per city, department, or product category.

REMEMBER GROUP BY changes the GRAIN of the output.

SQL STARTER

```
SELECT region, COUNT(*) AS n_orders,
       SUM(amount) AS revenue
FROM sales
WHERE amount > 0
GROUP BY region
HAVING COUNT(*) >= 3;
```

WATCH WHERE filters rows; HAVING filters groups.

SELF TEST Where do you filter groups with 3+ orders?

08 Conditional pivots

SUMMARIES

SPOT IT Show categories side by side as separate output columns.

REMEMBER CASE chooses a bucket; SUM adds that bucket.

SQL STARTER

```
SELECT region,
 SUM(CASE WHEN kind = 'A'
     THEN amount ELSE 0 END) AS a_sales,
 SUM(CASE WHEN kind = 'B'
     THEN amount ELSE 0 END) AS b_sales
FROM sales GROUP BY region;
```

WATCH Check NULLs, the ELSE branch, and row grain.

SELF TEST What creates separate A and B columns?

Patterns 9 + 10

Deduplication / CTE pipelines - Read cue, recall SQL, test yourself.

09 Deduplication

WINDOWS

SPOT IT Keep the latest or preferred row for each ID.

REMEMBER Rank within each ID, then KEEP row one.

SQL STARTER

```
WITH ranked AS (
 SELECT t.*, ROW_NUMBER() OVER (
  PARTITION BY user_id
  ORDER BY updated_at DESC, id DESC
 ) AS rn FROM records t
) SELECT * FROM ranked WHERE rn = 1;
```

WATCH Add a tie-breaker: updated_at may not be unique.

SELF TEST Why is DISTINCT alone insufficient?

10 CTE pipelines

BUILD / SEQUENCES

SPOT IT Turn a messy problem into named, smaller query steps.

REMEMBER WITH is a notebook of intermediate answers.

SQL STARTER

```
WITH daily AS (
 SELECT sale_date, SUM(amount) AS revenue
 FROM sales GROUP BY sale_date
)
SELECT sale_date, revenue
FROM daily WHERE revenue > 1000;
```

WATCH CTEs improve readability, not always runtime.

SELF TEST When are multiple named CTE steps useful?

Patterns 11 + 12

Correlated subqueries / Gaps and islands - Read cue, recall SQL, test yourself.

11 Correlated subqueries

RELATIONSHIPS

SPOT IT For each outer row, test a related condition.

REMEMBER The inner query refers BACK to an outer row.

SQL STARTER

```
SELECT c.customer_id
FROM customers c
WHERE EXISTS (
 SELECT 1 FROM orders o
 WHERE o.customer_id = c.customer_id
   AND o.amount > 100
);
```

WATCH EXISTS checks existence; inspect plan for speed.

SELF TEST Which outer alias does the inner query use?

12 Gaps and islands

BUILD / SEQUENCES

SPOT IT Find streaks of consecutive dates, IDs, or numbers.

REMEMBER For consecutive ints: value minus row_num = island.

SQL STARTER

```
WITH x AS (
 SELECT value,
  value - ROW_NUMBER() OVER (
   ORDER BY value) AS island_key
 FROM (SELECT DISTINCT value FROM nums) d
) SELECT MIN(value), MAX(value)
  FROM x GROUP BY island_key;
```

WATCH Deduplicate first; date math varies by SQL dialect.

SELF TEST Why does the subtraction find a streak?

Patterns 13 + 14

Date cohorts / Recursive queries - Read cue, recall SQL, test yourself.

13 Date cohorts

BUILD / SEQUENCES

SPOT IT Group users by first activity; follow later behavior.

REMEMBER FIRST event = cohort; LATER event = activity.

SQL STARTER

```
WITH firsts AS (
 SELECT user_id,
  DATE_TRUNC('month', MIN(event_ts)) AS cohort
 FROM events GROUP BY user_id
) SELECT cohort, COUNT(*) AS users
FROM firsts GROUP BY cohort;
```

WATCH This query counts cohort sizes, not retention.

SELF TEST What determines a user's cohort?

14 Recursive queries

BUILD / SEQUENCES

SPOT IT Walk an org chart or parent-child hierarchy.

REMEMBER ANCHOR starts; RECURSION follows; STOP guards.

SQL STARTER

```
WITH RECURSIVE org AS (
 SELECT id, manager_id, 0 AS depth
 FROM employees WHERE id=1
 UNION ALL
 SELECT e.id, e.manager_id, o.depth + 1
 FROM employees e JOIN org o
   ON e.manager_id=o.id WHERE o.depth<10
) SELECT * FROM org;
```

WATCH Depth limits help; visited IDs also prevent cycles.

SELF TEST Identify the anchor and recursive parts.

14 questions. No peeking.

Name each pattern. Only then look at page 11.

01 Best 3 salespeople in EACH department?

Pattern: _____

02 Customers who have never ordered?

Pattern: _____

03 Revenue accumulated day by day?

Pattern: _____

04 Latest status row for each user?

Pattern: _____

05 Employee name with manager name?

Pattern: _____

06 Orders per region, only regions with 5+?

Pattern: _____

07 Compare today's value with yesterday's?

Pattern: _____

08 Monthly new users by signup month?

Pattern: _____

09 Apple and banana sales in separate columns?

Pattern: _____

10 Group consecutive dates into streaks?

Pattern: _____

11 Find every descendant of a category?

Pattern: _____

12 Clients with an order above $100?

Pattern: _____

13 Make a query readable in 3 stages?

Pattern: _____

14 Give each student's rank inside the class?

Pattern: _____

The pattern recall key

Correct? Explain why. Wrong? Revisit only that pattern's card.

01 Top N by group
PARTITION resets the ranking for every gr

02 Anti-joins
NOT EXISTS means prove the match is absen

03 Running totals
SUM OVER + ORDER BY = rolling accumulatio

04 Deduplication
Rank within each ID, then KEEP row one.

05 Self joins
One table, two roles, two aliases.

06 Grouped aggregates
GROUP BY changes the GRAIN of the output.

07 Previous / next
LAG looks BACK; LEAD looks AHEAD.

08 Date cohorts
FIRST event = cohort; LATER event = activ

09 Conditional pivots
CASE chooses a bucket; SUM adds that buck

10 Gaps and islands
For consecutive ints: value minus row_num

11 Recursive queries
ANCHOR starts; RECURSION follows; STOP gu

12 Correlated subqueries
The inner query refers BACK to an outer r

13 CTE pipelines
WITH is a notebook of intermediate answer

14 Window ranking
WINDOW ranks rows without collapsing them

5-MINUTE SPACED REVIEW

Day 0: read all cards. Day 1: quiz. Day 3: write each SQL skeleton.
Day 7 + 14: repeat missed cards. Solve a new problem for each miss.